

Stock Portfolio Analyzer

Personal Stock Analyst Agent — built on Replit with Replit Agent

WEEK 1 BOOTCAMP SUBMISSION · PROJECT DOCUMENTATION

1. Project Overview

This project recreates the Week 1 “Stock Portfolio Analyzer” from the bootcamp handout entirely inside Replit, using Replit Agent as the vibe-coding partner. The result is an AI-powered **Personal Stock Analyst Agent**: a Streamlit application that takes a CSV of stock transactions (buys and sells) and produces FIFO-based cost-basis accounting, XIRR-based performance metrics, and a Groq-LLM-powered portfolio summary and chat assistant.

INPUT

A single CSV transaction log
— no manual entry anywhere
in the app

CORE LOGIC

FIFO cost basis, unrealized
and realized P&L, portfolio
XIRR

AI LAYER

Groq LLaMA-3.3-70B health
summary and context-aware
analyst chat

1.1 Stack

Component	Role in the project
Streamlit	Four-tab UI, upload widget, metric cards, chat interface
yfinance	Live and historical price lookups per ticker
Groq — LLaMA-3.3-70B	Portfolio health summary and the AI analyst chat
pandas · numpy · scipy	Transaction handling, FIFO lots, XIRR root-finding
Plotly	Allocation pie, cumulative cash-flow and monthly activity charts
uv	Package management and virtual environment — strictly no pip, venv, conda, poetry or requirements.txt
Replit Agent	The vibe-coding partner that generated and revised the codebase

1.2 The Four Tabs

Tab	Contents
1 — Data Upload	CSV-only transaction upload (ticker, date, transaction_type, quantity, price), validation, upload statistics, and the parsed dataframe
2 — Consolidated Portfolio	FIFO holdings table, allocation pie chart, total portfolio value metric, and a Groq-generated portfolio health summary with a concentration-risk callout
3 — Historical Performance	Total invested, total sell proceeds, realized P&L, total return, portfolio XIRR, cumulative cash-flow chart and monthly activity chart
4 — AI Analyst Chat	Groq-powered chat carrying full portfolio context, streaming responses, live-price lookups for “why is it down today” questions, and financial-advice disclaimers

2. Dataset Used

The sample transaction log used for the build — 13 BUY/SELL rows across three tickers spanning January 2024 to May 2026, which exercises repeat buys, partial sells and multiple FIFO lots per ticker:

```
ticker,date,transaction_type,quantity,price
AAPL,2024-01-03,BUY,10,185.07
MSFT,2024-04-03,BUY,10,420.3
AAPL,2024-07-03,SELL,7,215.8
TSLA,2024-10-02,BUY,13,246.53
MSFT,2025-01-02,SELL,3,400.38
AAPL,2025-04-01,BUY,6,169.23
TSLA,2025-07-01,SELL,10,260.86
MSFT,2025-10-01,BUY,7,410.69
AAPL,2026-01-02,SELL,4,195.58
TSLA,2026-03-18,BUY,9,180.31
MSFT,2026-04-19,SELL,5,425.82
AAPL,2026-05-21,BUY,5,190.27
TSLA,2026-05-25,SELL,8,175.71
```

Note that transaction_type arrives upper-cased here — the normalisation step case-folds it, which is exactly the mixed-case edge case the refined prompt called out. File: [stock-analysis-sample.csv](#).

The refined specification requires this shape to be validated on upload: required columns present, transaction_type restricted to Buy/Sell, positive numeric quantity and price, parseable dates, and normalized (upper-cased, trimmed) tickers.

3. Architecture

The application is a single Streamlit process with three utility modules behind four tab components. Prices come from yfinance, language output from Groq; everything else is computed locally from the uploaded CSV.

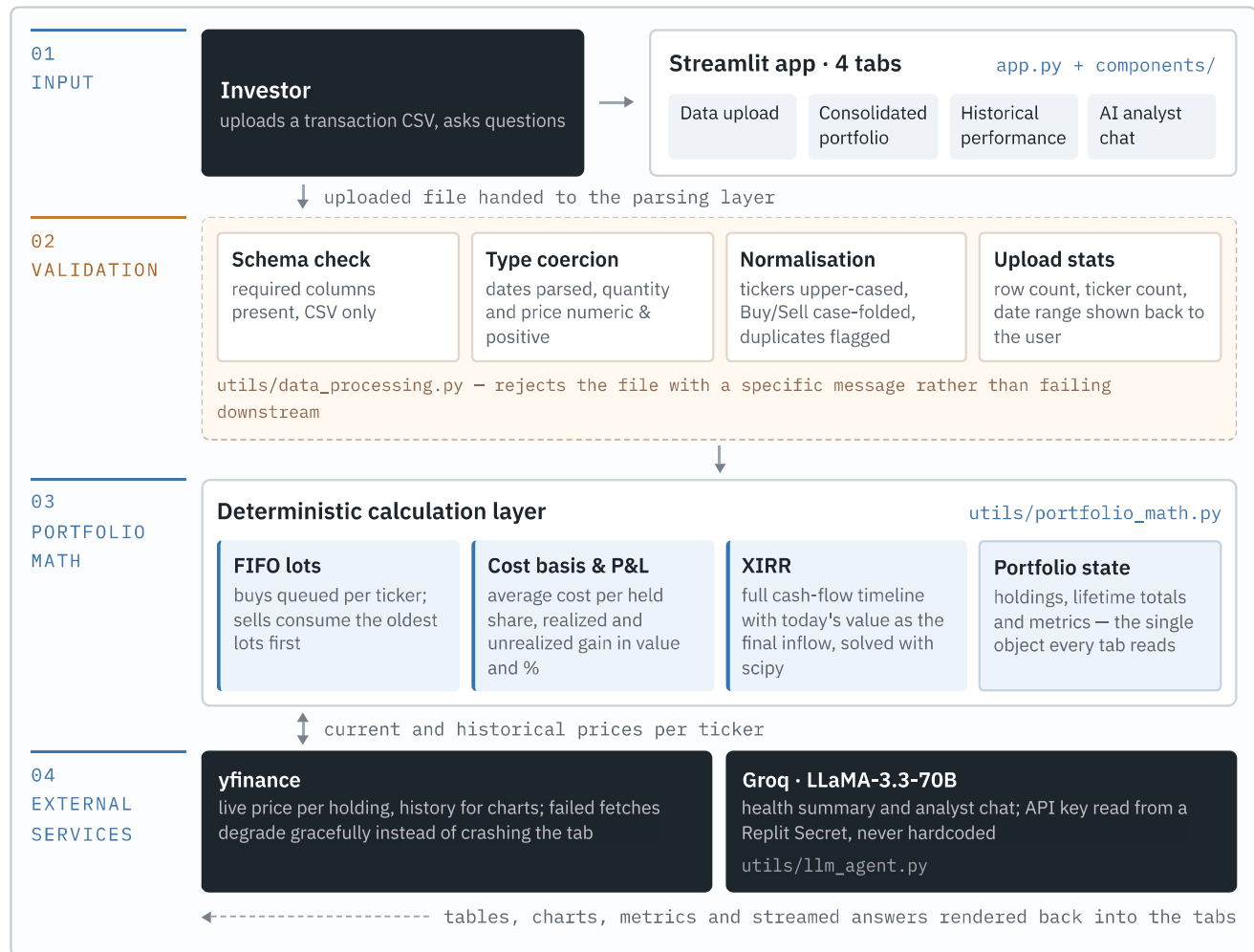


Figure 1 — Data path from CSV upload through validation and the deterministic portfolio-math layer to the two external services and back into the four tabs.

3.1 Project Structure

app.py	main application entry point
utils/	
├ data_processing.py	CSV parsing, validation, normalisation
├ portfolio_math.py	FIFO lots, cost basis, P&L, XIRR
└ llm_agent.py	Groq client, prompts, streaming chat
components/	one module per tab's UI logic
pyproject.toml · uv.lock	uv-managed dependencies (no requirements.txt)

4. Prompts Used During Vibe Coding

4.1 Initial Prompt

Build an AI-powered Streamlit app that acts as my personal Stock Analyst Agent to track and analyze my US stock portfolio. Use yfinance for live/historical stock data, Groq for LLM capabilities, and uv strictly for package management and virtual environment setup.

Create 4 Tabs as follows:

Tab 1: Data Upload

- Takes a CSV upload of my stock transaction history (NO manual entry).
- The CSV contains headers: ticker, date, transaction_type (Buy/Sell), quantity, price.
- Display the uploaded dataframe.

Tab 2: Consolidated Portfolio View

- Pie chart showing portfolio allocation based on current stock value.
- Metric card: Total Current Portfolio Value.
- Table displaying stock-wise breakdown: Ticker Symbol, Current Quantity, Avg Cost basis (use FIFO method to account for sells), Current Price, Unrealized Profit/Gain (Value & %).
- AI Feature: A small text container where Groq LLM provides a 2-3 sentence summary of portfolio health and concentration risk based on current holdings.

Tab 3: Historical Performance

- Metric cards for: Total Investment (lifetime), Total Sells (lifetime proceeds), Current Portfolio Value, Total Return, and XIRR for the entire portfolio timeline.

Tab 4: AI Analyst (Chat)

- AI Feature: An interactive chat interface powered by Groq. Pass the portfolio history, performance metrics, and current holdings as context so I can ask questions like "Summarize my historical trading performance" or "Why is my portfolio down today?"

Project Structure:

- app.py: main application entry point
- utils/: Folder for utility logic (data_processing.py, portfolio_math.py, llm_agent.py)
- components/: Folder for each tab's UI logic

Agent's response: it read the Streamlit skill, checked for an existing Groq integration (none found), installed Python and uv, then paused to ask for the Groq API key as a Replit Secret before proceeding.

4.2 Refined Prompt

Sent after reviewing the initial build, to lock in the exact calculation mechanics rather than leaving them to inference.

```
You are an expert Python full-stack developer and AI engineer. Build
a production-quality AI-powered Streamlit app that acts as my
personal US Stock Portfolio Analyst Agent.
```

The full prompt then specified, in detail:

- strict tech stack — Streamlit, yfinance, Groq, uv, Plotly, pandas/numpy, scipy;
- CSV-only input with full validation: required columns, Buy/Sell values, positive numeric quantity and price, valid dates, ticker normalization;
- Tab 1 upload statistics;
- Tab 2 FIFO holdings calculation — buy lots, sell reduction from the oldest lots, average cost basis, current price via yfinance, unrealized P&L, allocation pie chart, and a Groq-generated two-to-three sentence health summary with a concentration-risk callout;
- Tab 3 lifetime metrics — total investment, total sells, current value, total return, and XIRR over the full cash-flow timeline with today's value as the final positive cashflow, computed robustly with scipy;
- Tab 4 AI chat with full portfolio context, live-price lookups for “why is it down today” questions, and financial-advice disclaimers;
- Groq client requirements — API key from an environment variable, no hardcoding, configurable model, graceful error handling;
- the exact utils/ and components/ file structure;
- uv-only package management — pyproject.toml plus uv.lock, explicitly no pip, venv, conda, poetry or requirements.txt;
- README requirements;
- and a full list of edge cases: empty CSV, missing columns, invalid dates or types, oversell, fully sold positions, failed price fetch, missing API key, non-computable XIRR, duplicate rows, mixed-case input.

Agent rebuilt the app to this specification and confirmed all four tabs, the FIFO logic and the Groq integration were working.

5. Bugs Encountered & Fixes

These were the real, non-trivial issues hit during the build — the most valuable part of this walkthrough for other learners.

Bug	Cause and fix
1 • Port conflict	An old manually-created workflow was still holding port 5000, blocking the new Agent-managed workflow from starting. Fixed by removing the stale workflow from <code>.replit</code> and killing the leftover process.
2 • Startup syntax error	A broken, duplicated dict comprehension in <code>tab3_performance.py</code> — a malformed version sitting right above the correct one — crashed the import chain before the app could start. Fixed by removing the broken lines.
3 • Replit proxy / routing the deepest bug	Even with the app running correctly on port 5000, the preview returned 404: this Repl uses a managed-workflow/monorepo structure with a separate Node.js API server handling artifact routing, and the Streamlit app was never registered as a service at the root path (<code>/</code>) in that server's <code>artifact.toml</code> . Once registered, the page loaded a skeleton with no content, because Streamlit renders over a WebSocket and the Express proxy wasn't forwarding WebSocket upgrade requests. Fixed by explicitly wiring the proxy's upgrade handler to the HTTP server.
4 • KeyError on CSV upload <code>get_loc</code> misuse	<code>df.columns.get_loc("transaction_type")</code> returns an integer column position, not the column name — passing it into <code>sort_values()</code> raised a <code>KeyError</code> . Fixed manually by sorting on the column-name string directly.
5 • TypeError on Historical Performance <code>reset_index</code> misuse	A cumulative cash-flow Series called <code>.reset_index(columns=...)</code> , which is not a valid argument for a pandas Series. Fixed with <code>.reset_index(name="cumulative_cf")</code> .
6 • Missing GROQ_API_KEY	Tab 2's AI summary failed with “GROQ_API_KEY is not set” because the key had never been added to this Repl's Secrets — Secrets aren't shared across Repls even when reusing the same key value. Fixed by adding it under Tools → Secrets.
7 • Canvas vs. Preview confusion	Significant time went into distinguishing Replit's Canvas visual-editor overlay — which shows placeholder and loading-tip screens — from the actual dev Preview pane. An easy trap for first-time Replit users.
8 • Agent free-tier credit limit	The monthly free Agent-usage quota ran out partway through debugging. The remaining bugs (4 and 5) were fixed by editing source files directly in Replit's file editor, which costs no Agent credits.

6. Security Note

The Groq API key was inadvertently shared in plain text during the build process — it is visible in an early Agent “Add secrets” step and in a chat message. Any screenshot of the Secrets panel must have the key value redacted before sharing, and the key itself should be rotated in the Groq console.

7. Key Learnings

1. Splitting the build into an initial prompt plus a much more detailed refined prompt produced a far more correct result than trying to get everything right in one shot — the refined prompt's explicit FIFO, XIRR and validation spec was what actually locked in correct behaviour.
2. Agent will happily run ahead of scope — for example building three unrequested layout variants — when given an open-ended instruction like “all 3 layouts please”. Reining that back in with a clear correction prompt is the user's job.
3. Replit Secrets are per-Repl, not global: reusing the same API key value across projects still requires re-adding it as a Secret in each new Repl.
4. “It's running” and “it's visible in Preview” are not the same thing on Replit — routing and proxy configuration (`artifact.toml`, WebSocket forwarding) sits between the two and can silently break the preview even when the app underneath is healthy.
5. Manual file edits in Replit's built-in editor don't cost Agent credits — useful once a free-tier quota is hit, since simple, well-diagnosed bugs (such as a one-line `get_loc/reset_index` mistake) can be fixed directly without prompting Agent again.
6. Canvas mode and Preview mode look similar but serve different purposes — Canvas is a visual-editing overlay and can get stuck showing placeholder or loading content even when the real app is working fine underneath.

8. Screenshots

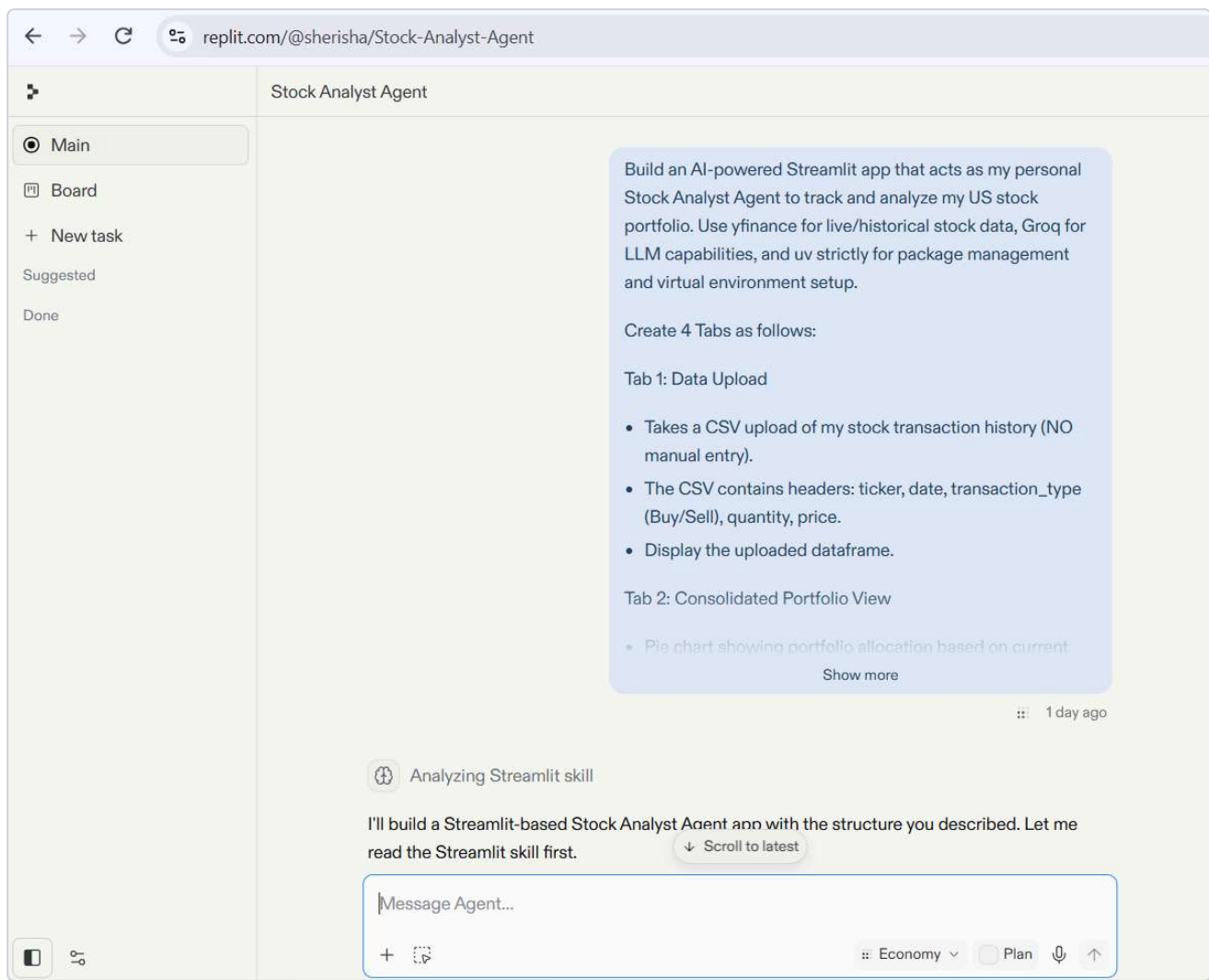


Figure 2 — Empty Repl right after creation.

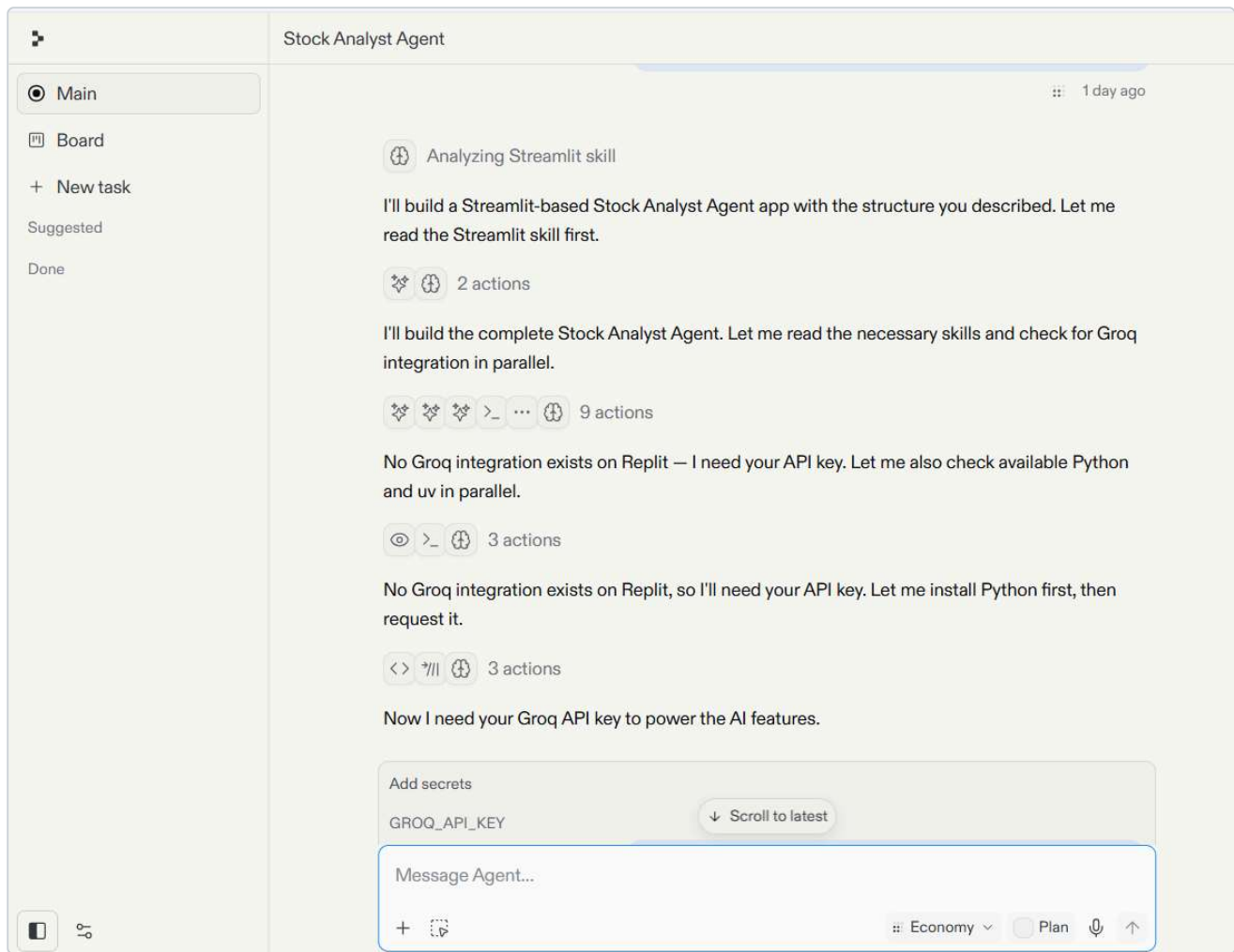


Figure 3 — Result after the initial prompt.

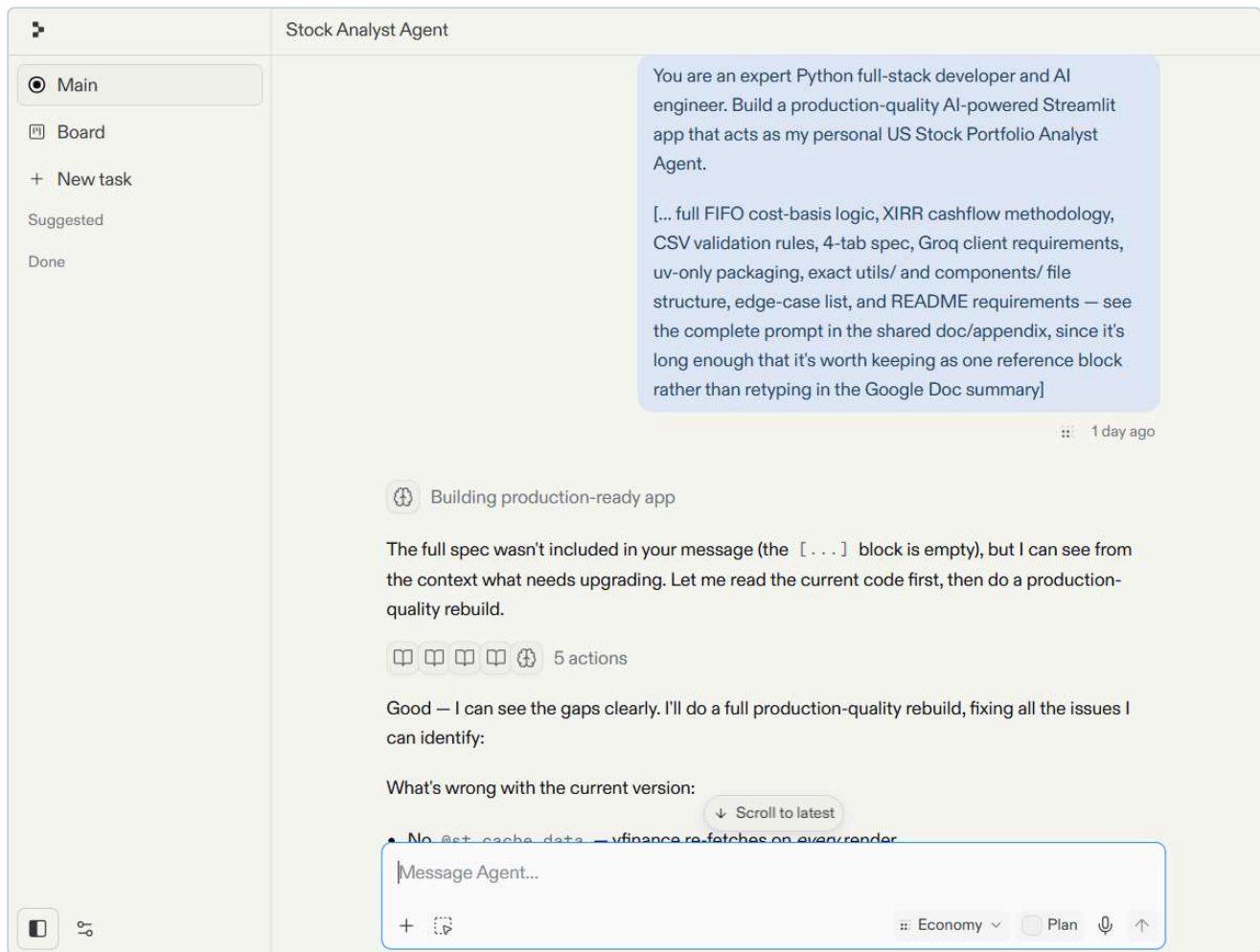


Figure 4 — Result after the refined prompt: all four tabs working.

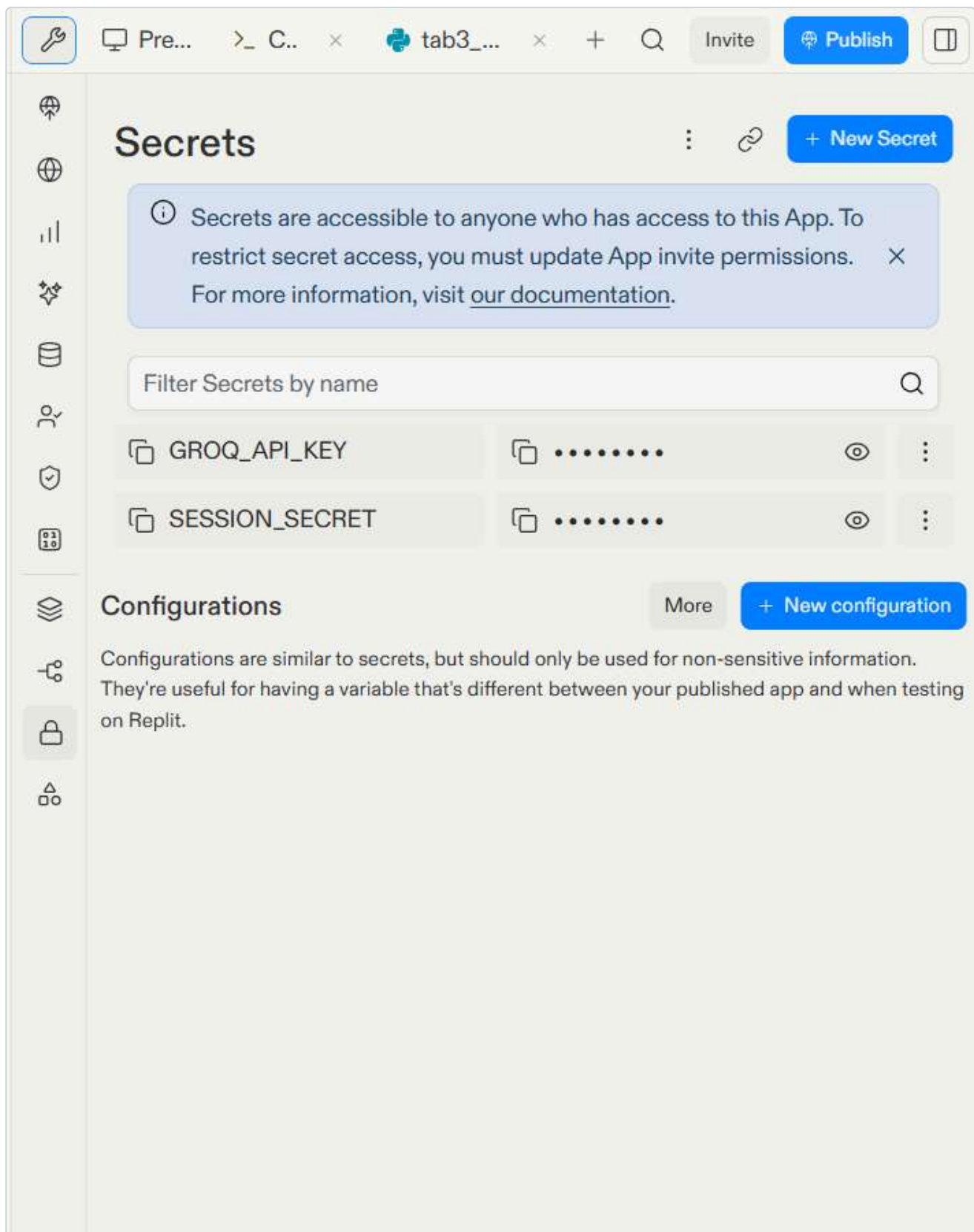


Figure 5 — Replit Secrets panel with GROQ_API_KEY added. Redact the key value before sharing this document.

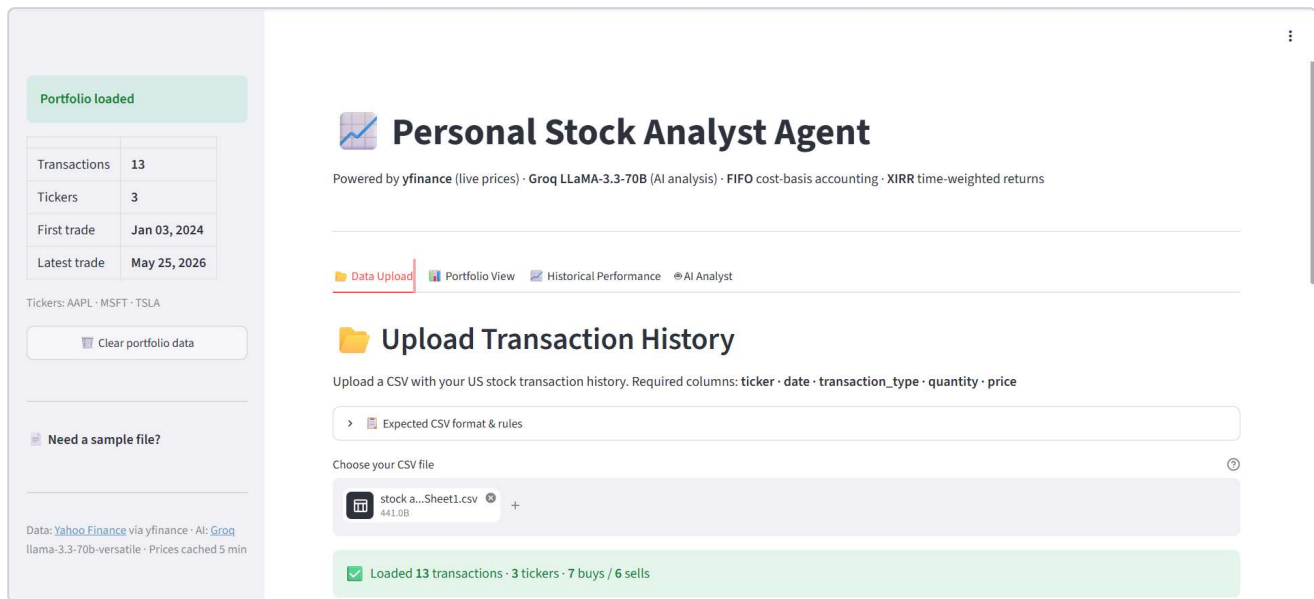


Figure 6 — Final working app: all four tabs, sample CSV loaded.

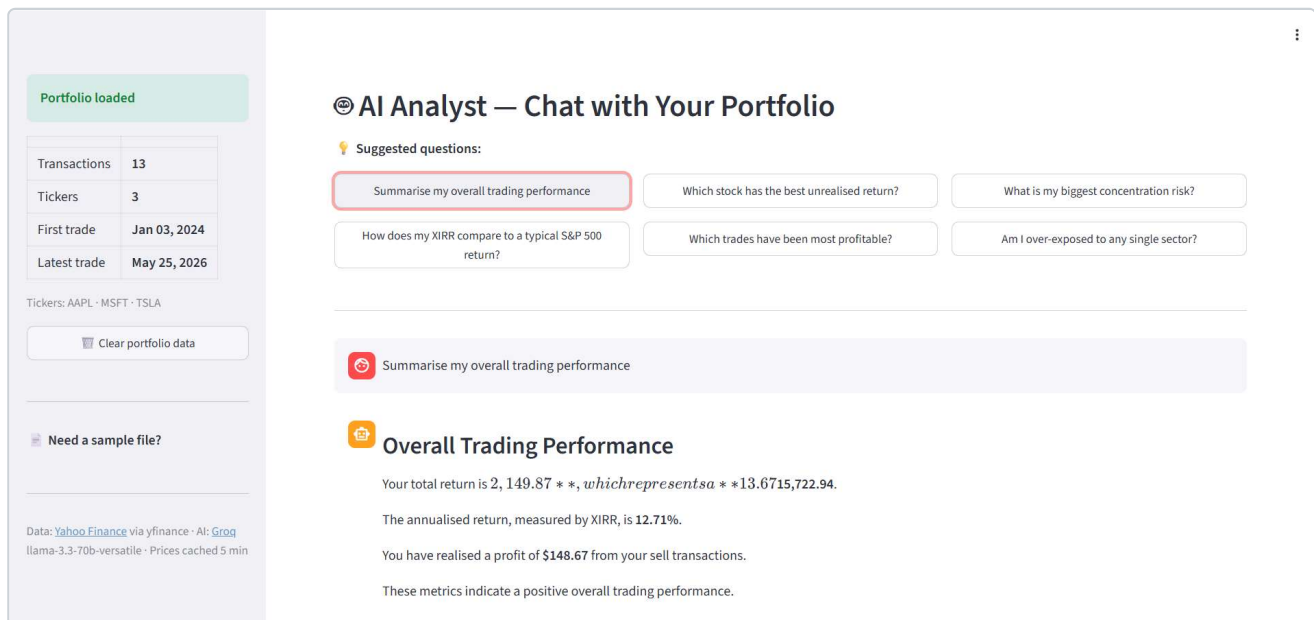


Figure 7 — AI Analyst Chat answering “Summarize my historical trading performance”.

9. Links

Replit project	replit.com/@sherisha/Stock-Analyst-Agent
-----------------------	---

Live app	60e414cf-5d99-40a8-9ea9-6c3126867a0a-00-fif914twsd08.sisko.replit.dev
-----------------	---

Dataset	
sample transaction log, 13 rows	stock-analysis-sample.csv

Video demo	loom.com/share/66332cf5edc047caadfd637a92569e8d
-------------------	---

End of Document